第2章：软件过程

- 学习目的和要求

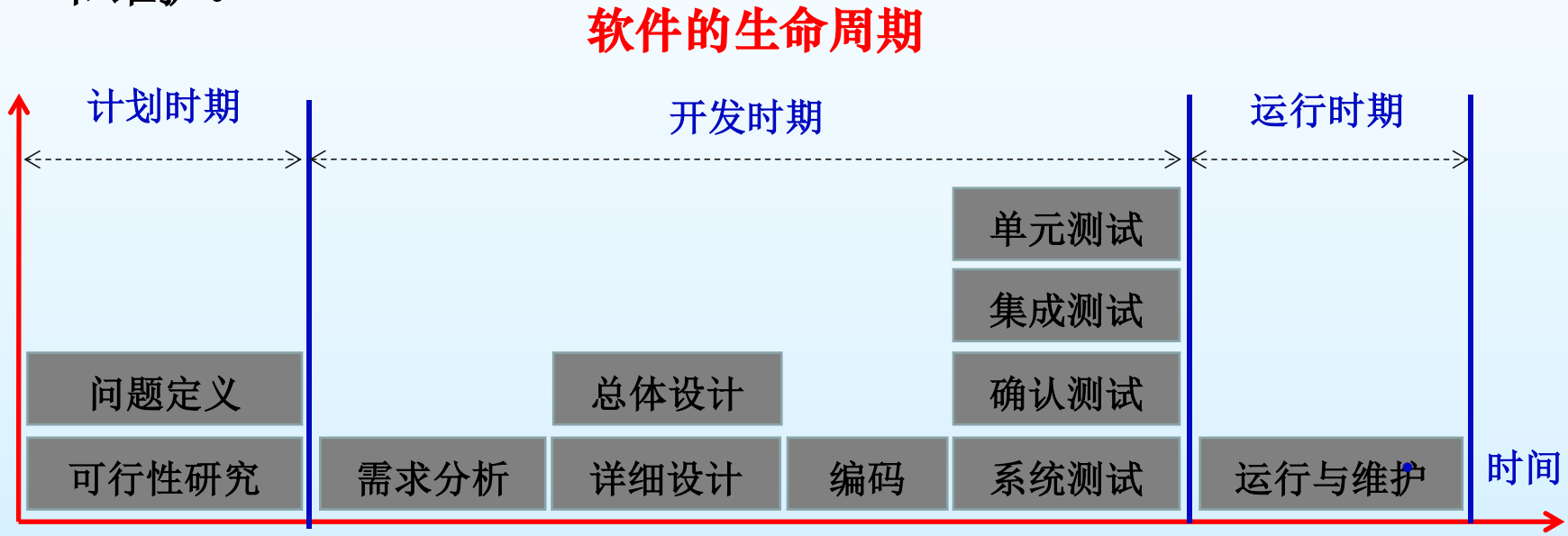
  - 软件工程过程—掌握
  - 常用过程模型—掌握
  - 应对变更过程模型—理解



软件的生存周期

- **定义**：软件生存周期是指一个软件从提出开发要求开始直到不再使用（报废）为止的整个时期。

- **划分**：软件生存周期包括三个时期：软件定义、软件开发及软件使用和维护。



软件的生存周期

计划时期

- **问题定义：** 确定要求解决的问题是什么？
- **可行性研究：** 确定在时间和资源的约束条件下，能否完成指定的任务？包括：技术可行性、经济可行性、法律可行性、社会可行性。若可行，则制定项目所需费用、资源、时间的开发计划。
- **需求分析：** 准确地确定“软件系统必须做什么”，即明确目标系统必须具备的功能和对系统的约束。

典型文档

1--项目计划书

描述将要完成的任务及其顺序，并估计所需要的时间及工作量。

2--软件需求规格说明书

描述将要开发的软件做什么，描述软件系统功能、性能、设计约束。

软件的生存周期

开发时期

- **概要设计**：确定在总体上应该怎样实现目标系统，包括系统的软件结构设计、接口设计和数据结构设计，制定集成测试计划。**--软件系统架构设计**
- **详细设计**：对软件结构中的模块进行精确描述（算法、数据结构等），形成可编程的程序模块，制定单元测试计划。**--功能模块界面设计、算法设计、流程设计、安全性设计等**
- **编码及模块测试**：把模块算法转换成特定程序设计语言的正确的、容易理解、容易维护的程序模块，对程序进行调试和单元测试，验证程序与详细设计文档的一致性。
- **测试**：发现和纠正程序中的错误，保证软件功能和性能符合需求规格说明书规定的要求。**包括集成测试、确认测试和系统测试。**

软件的生存周期

典型文档

3--概要设计说明书	描述软件的总体结构、设计及全局数据库和数据结构。
4--详细设计说明书	描述程序模块内部细节（算法、数据结构等），包括功能模块界面设计、算法设计、流程设计、安全性设计等。
5--软件测试计划	描述如何测试软件，使之确保软件应实现规定的功能，并达到预期的性能。
6--测试分析报告	在测试分析的基础上，对测试结果以及测试数据等加以记录和分析总结。包括测试目标与策略、测试环境、测试用例执行情况、缺陷报告、功能覆盖情况、非功能性测试覆盖、接口测试覆盖、风险评估、风险缓解措施、测试优化建议等
7--操作手册	按角色描述用户该如何使用软件

软件的生存周期

维护时期

软件投入使用后继续对软件的查错、纠错和改进。使系统持久地满足用户的需要。包括：

- ✓改正性维护：诊断和改正在使用过程中发现的软件错误；
- ✓适应性维护：修改软件以适应环境的变化；
- ✓完善性维护：根据用户的要求改进或扩充软件，使它更完善；
- ✓预防性维护：修改软件为将来的维护活动实现做准备。

典型文档

⇩

8--安装部署手册

软件系统安装部署手册是为了指导用户或系统管理员在实际环境中成功安装和部署软件系统，硬件要求、操作系统要求、软件依赖项、网络配置要求、安装步骤、配置与初始化、数据迁移与导入、系统启动与关闭等

软件工程的三要素

软件工程以关注软件质量为目标，包括过程、方法和工具三个要素。



软件过程：软件生存周期所涉及的一些列相关过程。通过工程化、标准化和形式化的方法管理软件的开发过程，从而改变基于手工作坊的软件生产方式，实现大规模生产。

任何软件生产都是有过程的，软件过程决定了软件的质量。

一、软件工程过程

1、软件过程定义

概念：软件过程是软件工程人员为了获得软件产品而在软件工具的支持下实施的一系列软件工程活动。

- 软件过程应明确定义：①团队人员的工作和职责；②所执行的活动及其顺序关系；③活动的内容和步骤。

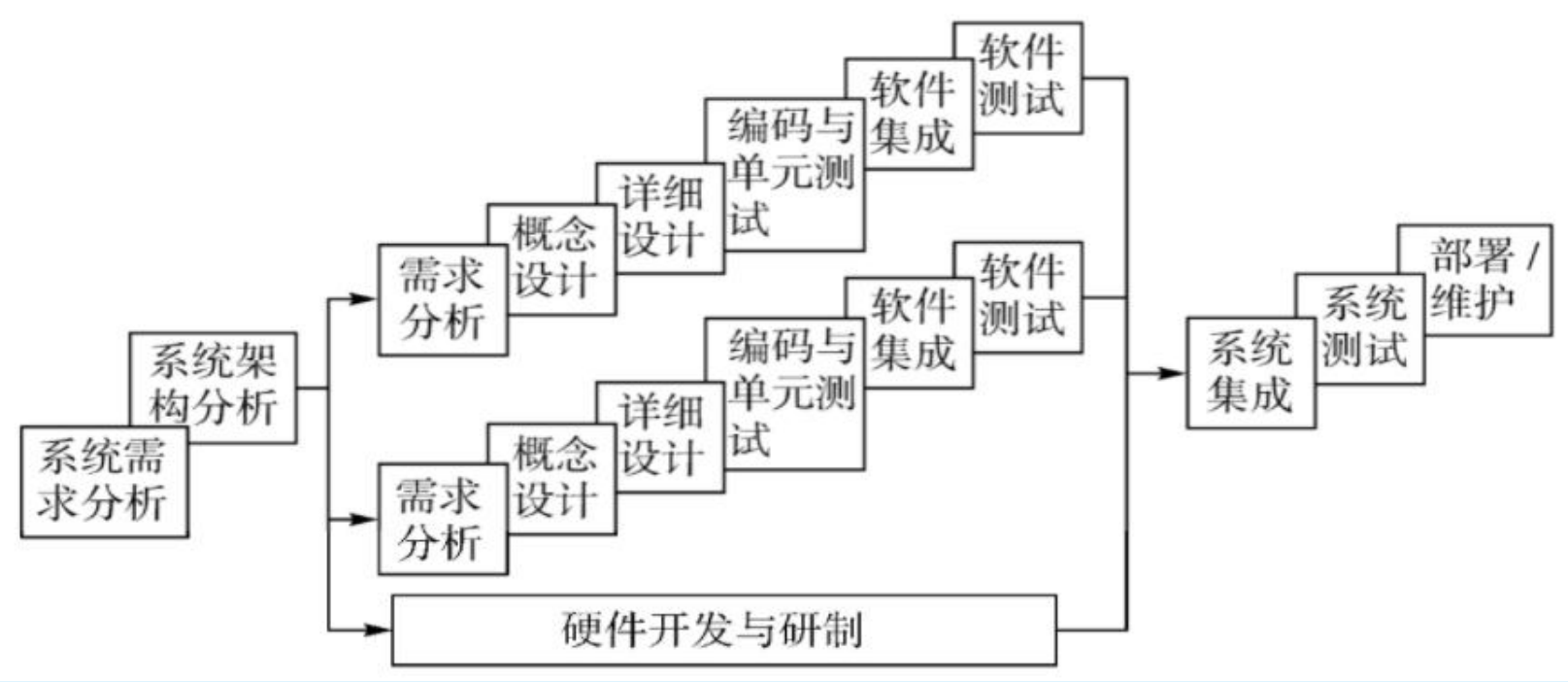


- 好处：
  - 通过提高可见性来降低开发风险；
  - 允许在项目进展过程中基于用户的反馈进行项目变更。

一、软件工程过程

2. 软件过程体系结构

- **主要软件过程元素：**需求分析、概念设计、详细设计、编码与单元测试、软件集成、部署维护



- **支持的软件过程元素：**配置管理、质量保证、文档编制、组间协调、同伴/进度检查

一、软件工程过程

3. 过程活动

软件过程是交织着技术、协作、管理等内容的一个活动序列。由于有许多不同类型的软件系统，因此，没有放之四海而皆准、适用于所有类型系统的软件工程方法，因此，也*没有放之四海而皆准的软件过程*。

不同企业所使用的过程取决于所开发的软件类型、软件客户需求以及开发软件人员的技能，但基本都包含以下4个最基本的软件工程活动：

□ 基本活动

1、**软件规格说明**—软件的功能及对软件运行的约束必须在这里进行定义；

2、**软件设计和实现**—软件设计和编程实现，必须生产符合规格说明的软件；

3、**软件有效性验证**—软件经过检查以保证它就是客户所需要的；

4、**软件进化**—软件随不同的客户和变化的市场需求而进行修改。

一、软件工程过程

3. 基本过程活动--①软件规格说明（需求工程）

是理解和定义系统需要提供哪些服务，以及找出开发和运行中的约束。需求工程是软件过程中的特别关键阶段，其**目标是生产一个达成一致的需求文档**。

需求工程过程如下：

①可行性研究。指明现有的软件、硬件技术能否实现用户对新系统的要求。从业务角度来决定系统开发是否划算以及在预算范围内能否开发出来。可行性研究应该是相对来讲花钱少且快速完成的。结果应该得出结论，该系统是否值得进行更细致的分析。

②需求导出和分析。这是一个通过对现有系统分析、与潜在用户和购买者讨论、进行任务分析等导出系统需求的过程。也可能需要开发一个或多个不同的系统模型和原型。这些都会帮助分析员了解所要描述的系统。

③需求描述。就是把在分析活动中收集的信息以文档的形式确定下来。在这个文档中有两类需求。用户需求是从客户和最终用户角度对系统需求的抽象描述；系统需求是对系统要提供的功能的详尽描述。

④需求有效性验证。该活动检查需求的现实性、一致性和完备性。在这个过程中，肯定会发现需求文档中的错误，必须加以改正以解决问题。

需求文档

一、软件工程过程

3. 基本过程活动--②软件设计和实现

开发的实现阶段是把系统描述成一个可运行的系统的过程，**设计过程通用模型**如下：

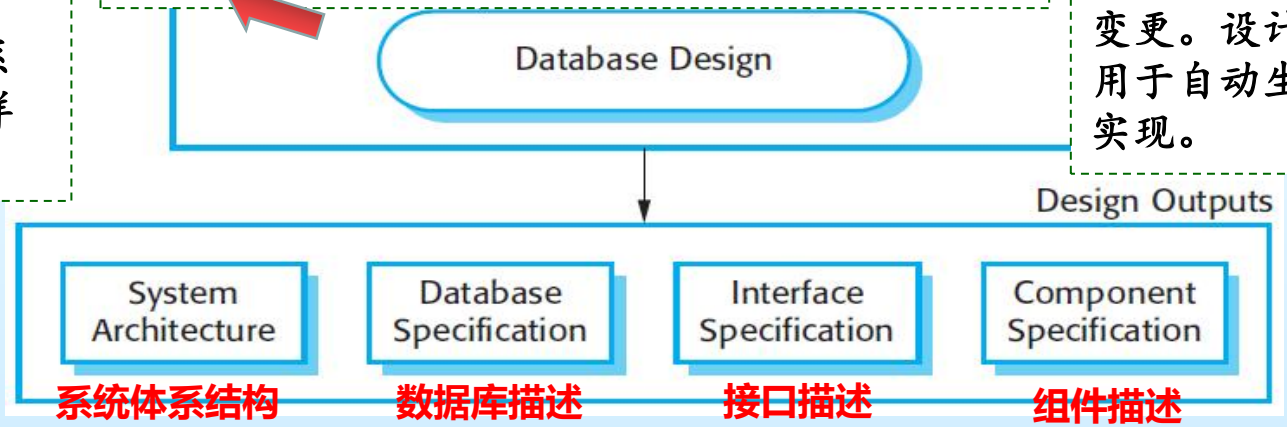
操作系统、数据库、中间件等软件支撑环境信息

②接口设计。定义系统组件间的接口。接口的描述必须是无二义的。在有精确接口定义的前提下，组件不必知道其他组件的具体实现即可使用它们。一旦接口描述达成一致意见，组件就可以并行设计和开发了。

①体系结构设计。识别系统总体结构、基本组件（有时候也叫子系统或模块）、它们之间的关系以及它们是怎样分布的。

③数据库设计。设计系统数据结构，以及如何在数据库中表示这些数据结构。此处的工作又一次取决于是否复用现有数据库或创建一个新的数据库。

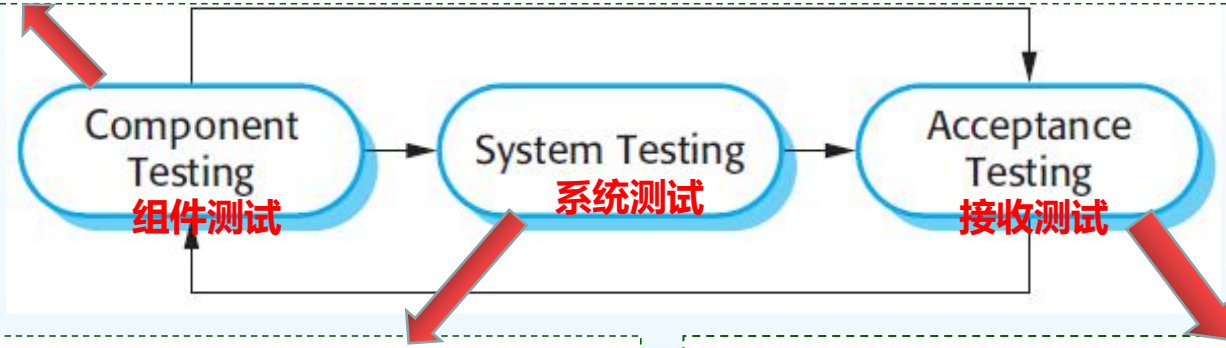
④组件设计。针对每个系统组件设计它的运行方式。这可能是对预期功能的一个简单声明，留给程序员去做具体的设计。也可能是对复用组件或是某个细化的设计模型所做的一系列的变更。设计模型可用于自动生成一个实现。



一、软件工程过程

3. 基本过程活动--③软件有效性验证

①组件（或单元）测试。由开发系统的人员对组成系统的组件进行测试。每个组件都单独测试，而不受其他系统组件的影响。组件可能是简单的实体，如一个函数或对象类、或者是这些实体的一个相关集合。



②系统测试。集成组件形成完整的系统。这个过程主要是关注无法预测的组件间交互和组件界面问题所引发的错误。也关注系统是否满足了功能上和非功能上的需求，并测试系统的总体特性。对于大型系统来说，这可能是一个多阶段的过程，需要对组件所构成的子系统进行测试，然后测试由这些子系统所构成的最终系统。

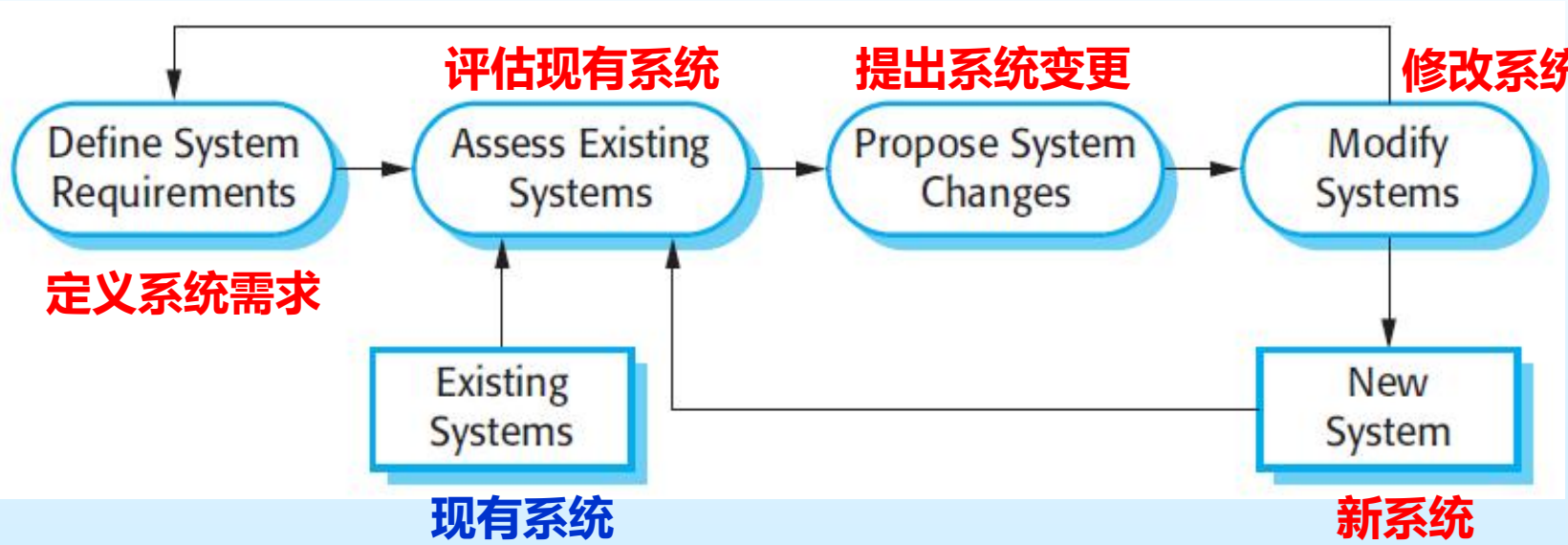
③接收测试。这是系统在接受并运行之前进行的最后阶段测试。这个阶段不再是用模拟数据来测试系统，而是用客户提供的真实数据测试系统。真实数据能以不同的方式测试系统，所以能暴露出系统需求定义中的错误和遗漏。接收测试还能发现系统需求中的类问题，即系统的设施不能满足用户的需要或者系统性能是无法接受的。

一、软件工程过程

3. 基本过程活动--④软件进化

自有软件开发以来，就有软件开发过程和软件进化（软件维护）过程之分。现在看来，这种划分越来越不恰当了。现在完全从头开发的系统很少，将软件系统的开发和维护看成一个连续体更有意义。

因此，不再将软件工程看作是开发和维护两个完全独立的过程，而是将其看成一个进化过程，进化式过程如下：



一、软件工程过程

4、过程描述

□ 过程描述： 过程中的活动及其顺序，以及每个活动中的产品交付物、角色和前后置条件。

1、产品交付物—这是软件过程活动的结果。如体系结构设计的产出物是体系结构模型。

2、角色—反映了人在软件过程中的职责。如项目经理、配置经理、程序员等。

3、前置和后置条件—是指在一个过程活动执行前或执行后，或者产品生产前或后必须满足的条件。如在体系结构设计之前，一个前置条件是客户已经认可了需求分析，一个后置条件是描述体系结构的UML模型进行评审。

二、常用过程模型



不同阶段划分方式以及各阶段要求的不同标准就形成了不同的模型，我们称之为软件过程模型。

软件过程模型定义

软件过程模型是对实际过程的抽象描述。包括软件过程的活动、软件产品以及参与人员的不同角色。

通俗地讲，按软件类型的特点、软件公司的水平情况的不同，在软件开发过程中划分不同的阶段，每个阶段的要求和标准不一样，就形成了不同的软件过程标准，称之为软件过程模型。

二、常用过程模型

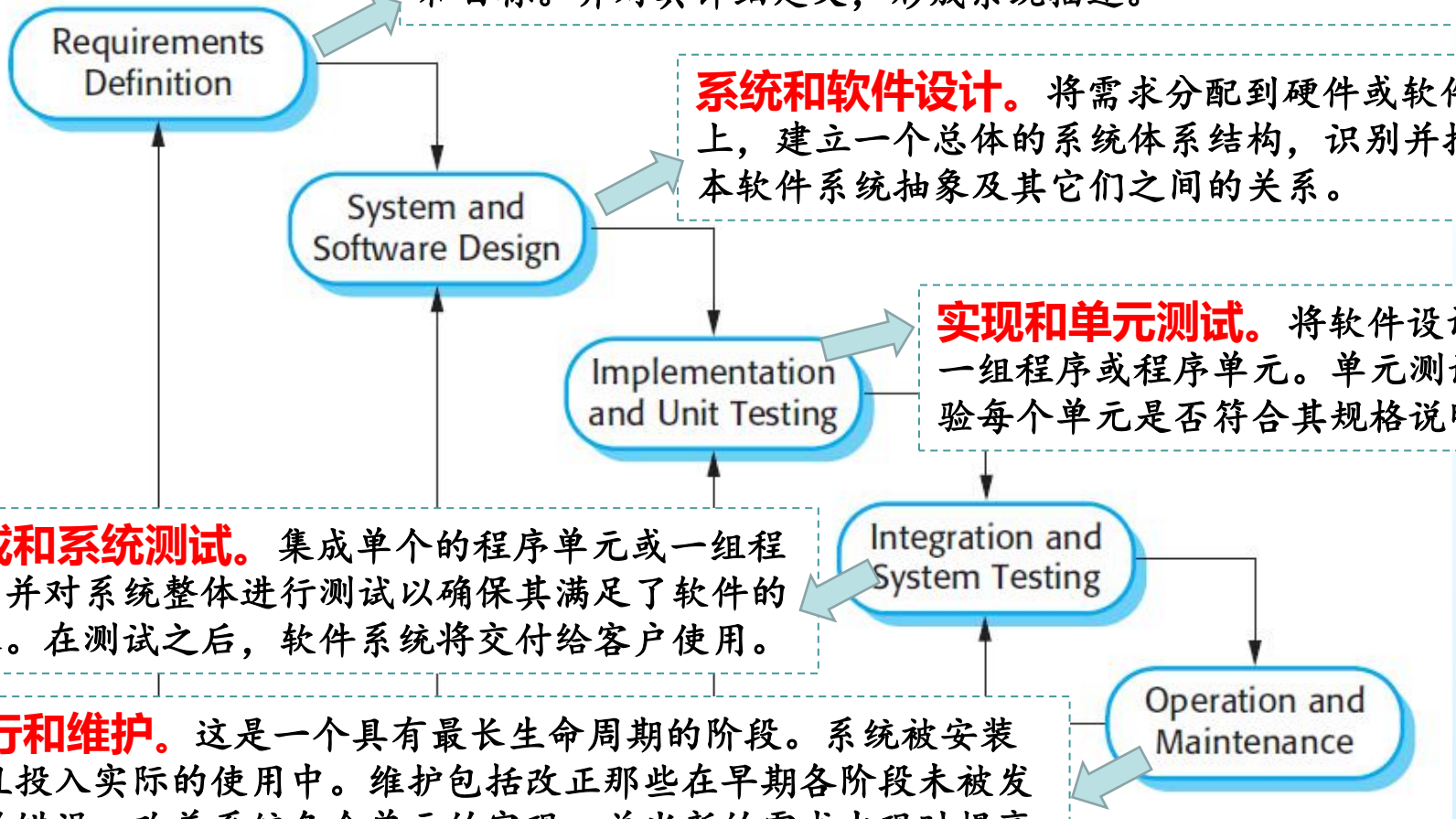
软件过程模型是软件过程的简化表示。每个过程模型都从侧面表现软件过程，所以只提供过程的部分信息。

□ 3种最常用的过程模型（也称：过程范型）

- **瀑布模型。**该模型将基本的过程活动（规格说明、开发、有效性验证和进化）看成是一些界限分明的独立的过程阶段，例如，需求描述阶段、软件设计阶段、实现阶段、测试阶段，等等。
- **增量模型。**该方法使得规格说明、开发和有效性验证活动交织在一起。系统的开发是建立一系列的版本（增量），每个版本添加部分功能到先前的版本中。
- **基于组件的开发模型（集成与配置）。**该方法依赖于可复用的构件或系统。系统开发过程中关注在新的使用环境中配置这些构件并将它们集成为一个系统。

二、常用过程模型

1. 瀑布模型



需求分析和定义。通过咨询系统用户建立系统的服务、约束和目标。并对其详细定义，形成系统描述。

系统和软件设计。将需求分配到硬件或软件系统上，建立一个总体的系统体系结构，识别并描述基本软件系统抽象及其它们之间的关系。

实现和单元测试。将软件设计实现为一组程序或程序单元。单元测试就是检验每个单元是否符合其规格说明。

集成和系统测试。集成单个的程序单元或一组程序，并对系统整体进行测试以确保其满足了软件的需求。在测试之后，软件系统将交付给客户使用。

运行和维护。这是一个具有最长生命周期的阶段。系统被安装并且投入实际的使用中。维护包括改正那些在早期各阶段未被发现的错误，改善系统各个单元的实现，并当新的需求出现时提高系统的服务能力。

二、常用过程模型

1. 瀑布模型

□ 瀑布模型的特点：

- **计划驱动**——理论上，在开始工作之前，你必须对所有的过程活动制订计划并给出进度安排。
- **阶段分明**——每个阶段的结果是一个或多个经过核准的文档。直到上一个阶段完成，下一阶段才能启动。
- **反复迭代**——各个活动之间存在交互和反复。
- **适度冻结**——需求的冻结有助于活动的开展。
- **文档为王**——阶段成果和交互的唯一凭据。

瀑布模型的一个重要变体是形式化的系统开发，其中会创建系统规格说明的数学模型。该模型会通过使用保持一致性的数学变换精化为可执行的代码。

二、常用过程模型

1. 瀑布模型

瀑布模型的优缺点分析

适用： 在开发早期阶段软件需求被完整确定。

缺点：

- 现实中客户很难明确地描述软件的需求，由于需求无法确定，就无

瀑布模型适合于下列类型的系统：

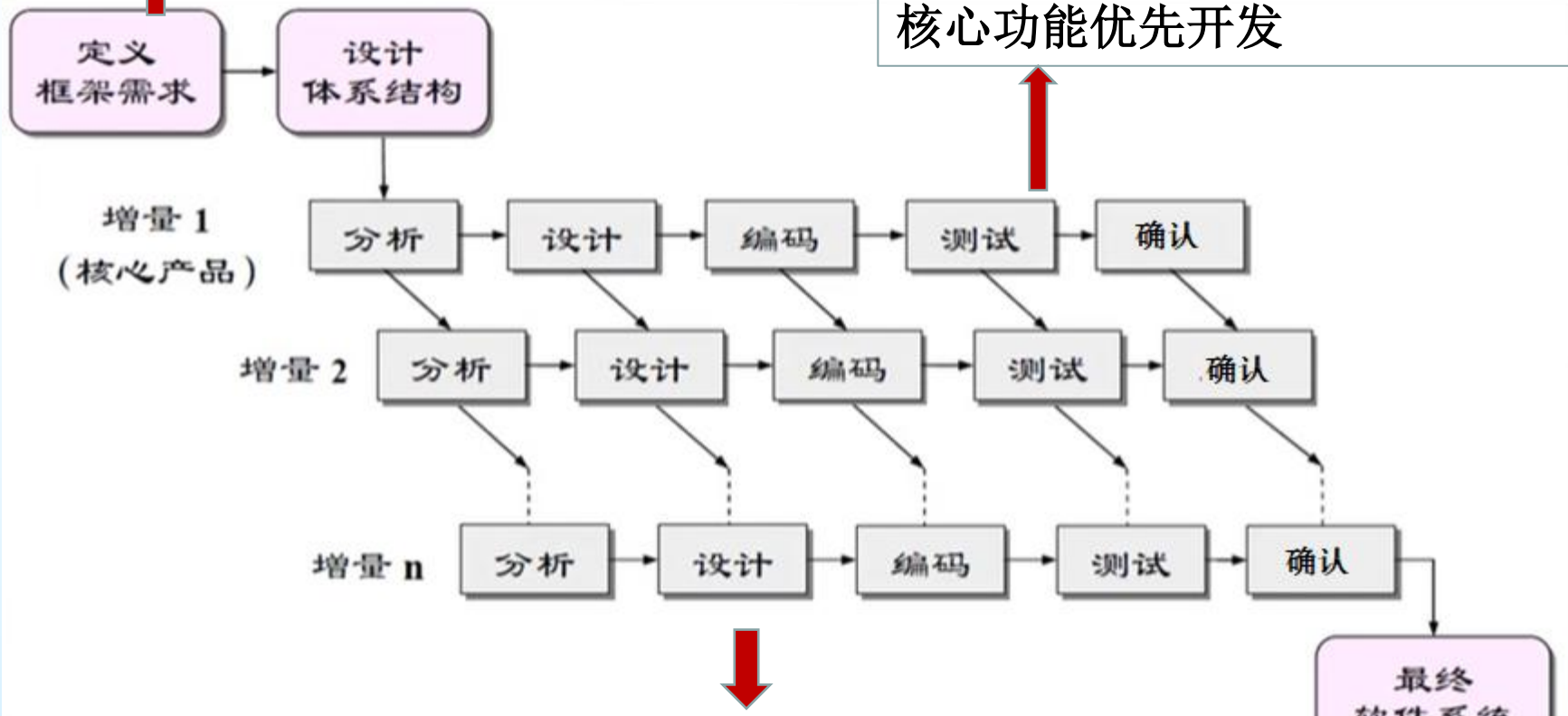
1. 嵌入式系统。其中软件必须与硬件系统连接和交互。由于硬件不灵活，将软件功能的决策推迟到开发时通常不可行。
2. 安全攸关性系统。要求对软件规格说明和设计的安全性和信息安全进行全面的分析。这些系统中，规格说明和设计必须完整。
3. 大型软件系统。由多家合作企业共同开发的工程化系统的一部分。系统中的硬件可能使用相似的模型开发，由于涉及多家企业，需要完整的规格说明以使不同的子系统可以独立开发。

二、常用过程模型

2. 增量过程模型（增量式开发）

定义系统框架层面的需求

一个功能一个增量的形式开发，核心功能优先开发



每个功能都要经过分析、设计、编码、测试、交付5个阶段

二、常用过程模型

2. 增量过程模型（增量式开发）

增量式软件开发，是敏捷方法的一个基本部分，对于商务、电子商务和个人系统来说更加适合。用户可以不断地、最快地看到软件可运行的中间版本，对于抢占市场入口的软件特别适用。

□ 增量式开发的特点：

- **规格说明、开发和有效性验证等活动不是分离的而是交织在一起**
- **并非提前制定出完整的问题解决方案，而是逐步地逼近解决方案，当意识到错误的时候更方便更快捷地回溯**
- **系统的每一个增量或版本包括用户需要的一部分功能（按重要性排序）**
- **需要开放的体系结构，不适合大型的、生命周期很长、不同的团队联合开发的系统**

二、常用过程模型

2. 增量过程模型（增量式开发）

□ 增量式开发的优点：

- **降低了适应用户需求变更的成本。**重新分析和修改文档的工作量较之瀑布模型要少很多。
- **在开发过程中更容易得到用户对于已做的开发工作的反馈意见。**用户可以评价软件的现实版本，并可以看到已经实现了多少。
- **使更快地交付和部署有用的软件到客户方变成了可能，**虽然不是所有的功能都已经包含在内。相比于瀑布模型，用户可以更早地使用软件并创造商业价值。

□ 增量式开发的缺点：

- **过程不可见。**管理者需要通过经常性的可交付文档来把握进度，如果系统开发速度太快，要产生反映系统每个版本的文档就很不划算。
- 伴随着新的增量的添加，**系统结构在逐渐退化。**除非投入时间和金钱用在重构系统结构上以改善软件，否则定期的变更会损坏系统的结构。随着时间的推移，越往后变更系统越困难，而且成本也将逐渐上升。

二、常用过程模型

2. 增量过程模型（增量式开发）

口增量式开发的适用情况

- **在开始开发时，需求很明确，且产品还可被适当地分解为一些独立的、可交付的软件（构造增量：Build increments．若一个增量不需要交付给客户的话，那么这样的增量通常称为一个“构造”(Build)。若增量需要被交付的话，那么它们就被认为是发布版本(Released version)。）；**
- **在开发中，期望尽快提交其中的一些增量产品。**

例如：

一个数据库系统，它必须通过不同的用户界面，为不同类型的用户提供不同的功能。在这一情况下，首先实现完整的数据库设计，并把一组具有高优先级的用户功能和界面作为一个增量；以后，陆续构造其它类型用户所需求的增量。

二、常用过程模型

2. 增量过程模型（增量式开发）

附：微软“同步-稳定的产品开发模型”



Microsoft®

- 将项目分为若干个里程碑阶段
- 定义稳定、灵活的体系结构，并为构件和子系统的开发提供统一的接口
- 开发构件，维持一个可发布的系统版本

可以准确把握项目进展情况

增强开发人员的信心和成就感

可以随时根据市场情况及时作出调整

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

■ 长期以来的软件开发状况

① 多数软件都是针对某个具体的应用系统从头进行开发的；

②导致：出现了大量的同类软件重复开发，造成大量人力、财力的浪费，而且软件的质量也不高。

■ 对比：汽车工业的生产模式

① 在已有的部件基础上通过组装进行生产：有专门的部件生产工厂，汽车设计者在设计中选择市场上已有的合适的部件；

② 避免了大量的重复劳动，提高了产品质量和生产效率，促进分工合作。

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

上万个零部件，70%以上由协作厂商完成

- 合适的时间
- 合适的东西
- 合适的数量



生产节拍：30秒～2分



自2000年开始，关注复用已有软件的软件开发已经得到广泛的使用。**面向复用的方法依赖于一个可复用的软件构件库以及一个用于构件组装的集成框架。基于组件的软件工程（component-based software engineering,CBSE）是强调使用可复用的软件组件来设计和构造基于计算机的系统。**

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

软件的生产过程划分为三类不同的生产车间，即应用构架生产车间、构件生产车间和基于构件、构架复用的应用集成(组装)车间，从而形成软件产业内部的合理分工，实现软件的工业化生产。

软件开发人员被划分成三类：构件生产者、构件库管理者和构件复用者。这三种角色所需完成的任务是不同：



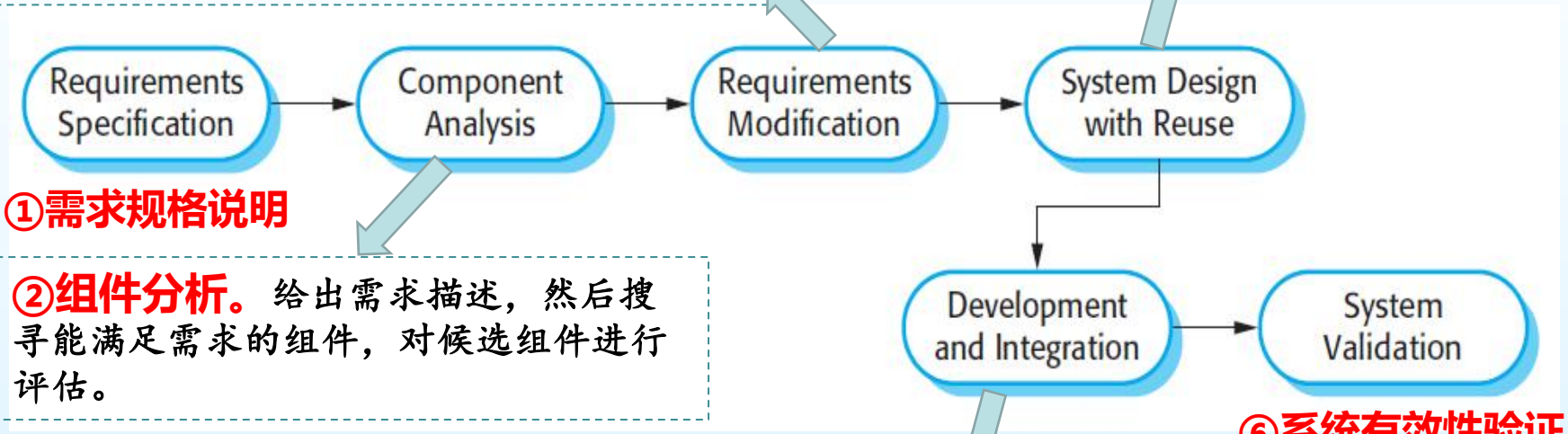
- **构件生产者：**负责构件的生产、描述；
- **构件库管理者：**负责构件分类以及构件库的管理工作；
- **构件复用者：**负责进行基于构件的软件开发，包括构件查询、构件理解、适应性修改、构件组装以及系统演化。

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

③需求精化。利用所发现的可复用组件和应用的信息对需求进行精化，对需求进行修改以反应可用的组件，系统规格说明被重新定义。如果无法修改，重新组件分析寻找可替代方案。

④面向复用的系统设计。设计系统的框架或者重复使用一个已存在的框架。如果一个满足需求的成品是可用的，那么可以对该系统进行配置后使用以创建新系统。



①需求规格说明

②组件分析。给出需求描述，然后搜寻能满足需求的组件，对候选组件进行评估。

⑥系统有效性验证

⑤开发和集成。当组件不能买到时就需要自己开发，然后集成这些自己开发的组件和现货组件，使之成为一个整体。

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

■ **框架：**框架是一个应用程序的半成品，提供可重用的公共结构，它封装了某领域内处理流程的控制逻辑。



框架是对问题的部分解决，也是用户集成构件的架构。框架描述了应用系统中所有的构件、构件之间的交互、连接件以及如何将构件和连接件结合的规则。

➢**Gartner Group认为：**应用开发的未来就是在于提供一开放体系结构，以方便构件的选择、组装和集成。

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

■ 框架的优势

- 不用再考虑公共问题，框架已经帮我们做好了。框架集成了前人的经验和智慧，可以帮助“新手”写出稳健、性能优良而且结构优美的高质量程序。
- 可以专心在业务逻辑实现上，保证核心业务逻辑开发质量。
- 结构统一、易于学习、维护。

■ 应用框架的要点

- 一般情况下，不要尝试去修改框架的核心代码；框架定义好了约束，开发者在规约下编写自己的业务代码，明白自己的业务代码放在何处是最合理的，然后框架把业务代码粘合起来。

二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

- 技术、构件、框架和系统的区别

概念	说明	举例
技术	解决某一类问题的方法	JSP技术、JDBC技术、XML技术
构件	应用程序里可重用的“零件”	分页构件、控制器构件、视图构件
框架	一系列构件按照一定的结构组合在一起作为系统开发的平台	Struts、Spring、Hibernate、EJB
系统	实现完整功能的应用程序	物流管理系统、销售系统

- 软件框架提供了创建具体应用的基本构造单元。
- 软件框架是一个部分完成的软件（子）系统，它将要被进一步实例化。

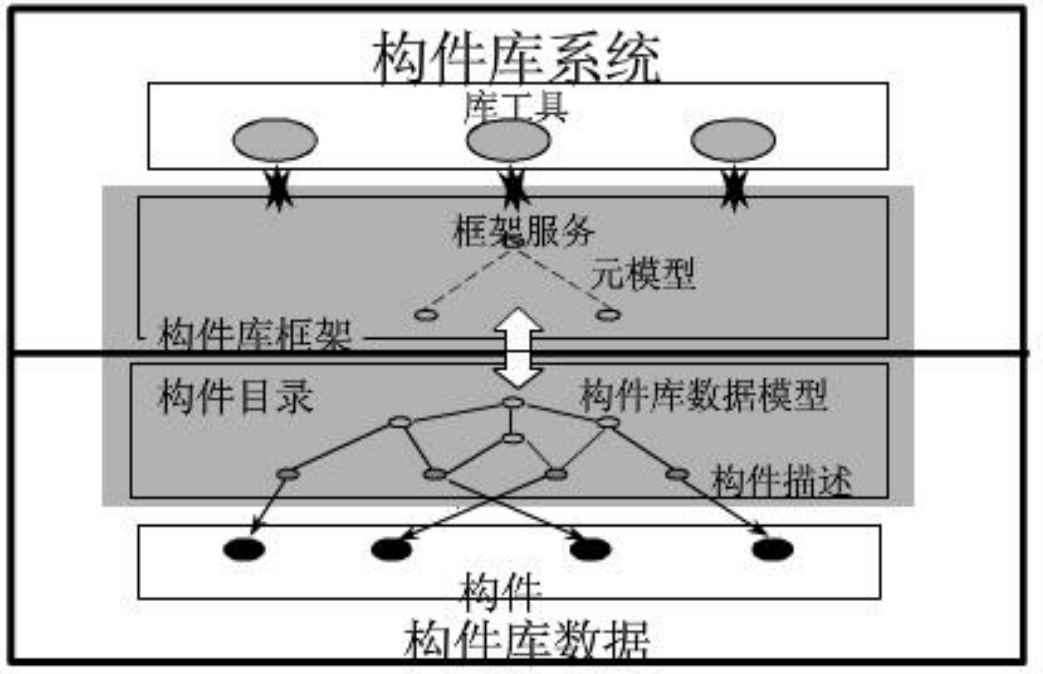
二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

3种类型的软件组件可用于面向复用的过程：

- 通过标准服务开发的Web服务，可用于远程调用。
- 对象的集合，作为一个包和组件框架，如.NET或者J2EE等集成在一起。
- 独立的软件系统，通过配置在特定的环境下使用。

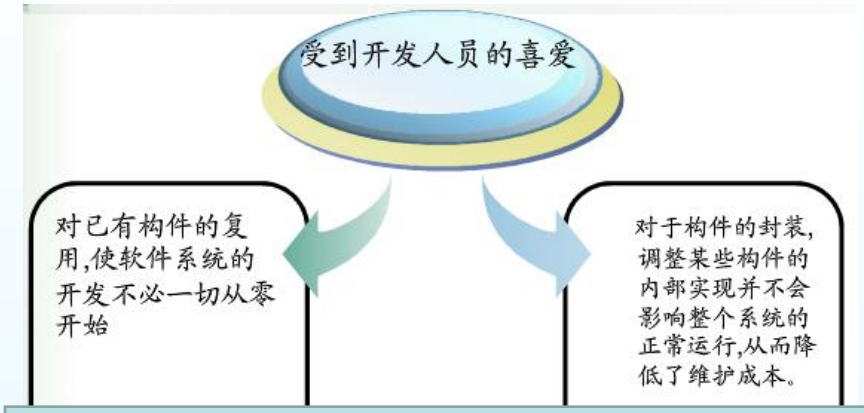
良好的构件分类有利于构件库的组织和管理，在基于构件的系统开发中，无论是构件的存储还是构件的检索，都依赖于构件的组织管理方式。

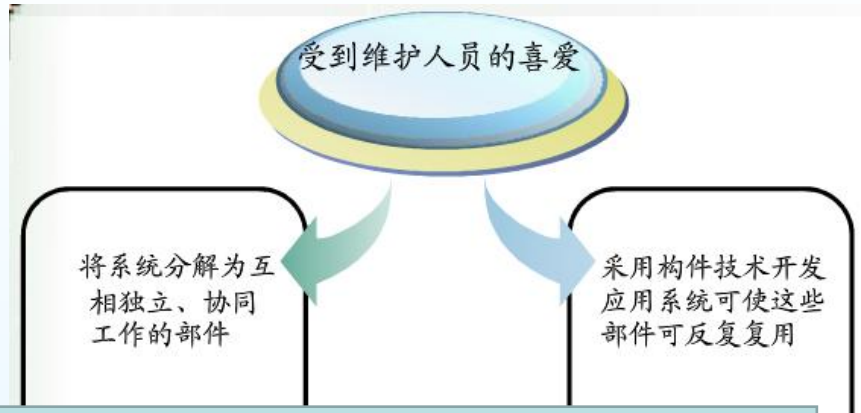


二、常用过程模型

3. 面向复用的软件工程（基于组件的开发模型）

- 基于复用的软件开发的优点：





没有放之四海皆准的软件过程，正确的过程取决于客户和管理需求、软件适用所处的环境以及所开发的软件类型。

例如：对于安全攸关的系统，需要一个非常结构化的开发过程，通常使用瀑布模型，其中要保存详细的记录。对于业务系统，由于需要适应快速变化的需求，因此一个更加灵活、敏捷的过程可能更好，通常使用配置和集成已有系统来构造所需功能。

三、应对变更过程模型

对于所有大型项目来说，变更是无法避免的。系统需求是在变化的，因为采购系统的工作要屈从于外部压力和管理权力的更替。当有新的技术可用的时候，新的设计和实现方法就会出现。因此，不管采用什么样的软件过程模型，能在软件开发过程中及时处理变更是必要的。

- **软件变更是必然事件。**
- **某些变更是可以避免的**——软件过程包括一些能够在重大返工发生之前预测变更的活动。例如，原型系统的开发，要先给客户看系统的一些重要特征。客户可以试用原型，并在花费高额的软件生产成本之前重新定义需求。
- **某些变更是可以容忍的**——所设计的过程使得变更以相对较低的成本得到处理。这通常需要增量开发。提出的变更可能是在还没有开发的增量上实现。假如不是这样的，只需要更改单个增量（系统的一小部分）来适应变更。

三、应对变更过程模型

应对变更的过程模型，也称为演化过程模型，常见的包括：原型开发、增量式交付和螺旋模型。

1、原型开发（原型开发范型）

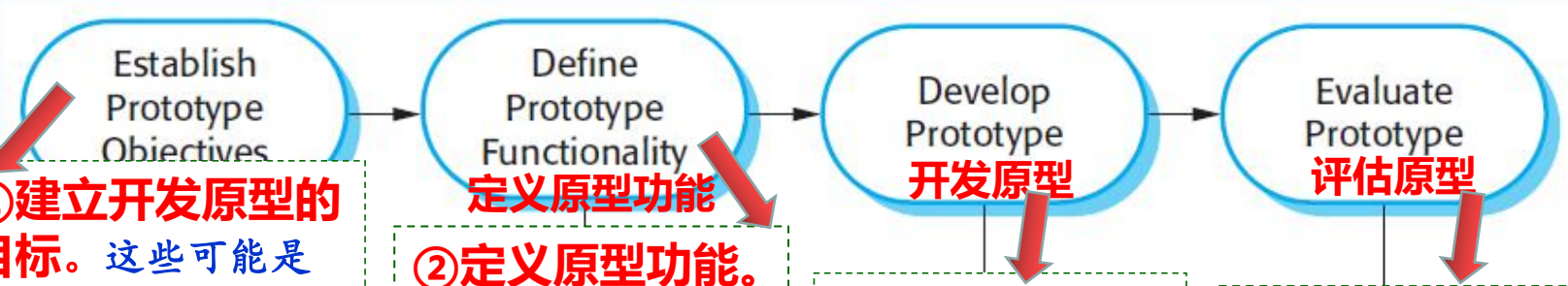
很多时候客户定义了软件的一些基本任务，但没有详细定义功能和特性需求。另一种情况下，开发人员可能对算法的效率、操作系统的适应性和人机交互的形式等没有把握。**在这些情况和类似的情况下，采用原型开发范型是最好的解决办法。**

虽然原型可作为一个独立的过程模型，但是更多时候是作为一种技术，可以在本章讨论的任何一种过程模型中应用。不论人们以什么方式应用，**当需求很模糊时，原型开发模型都能帮助软件开发人员和利益相关者更好地理解究竟要做什么。**

三、应对变更过程模型

1、原型开发（原型开发范型）

原型是一个软件系统的最初版本，用于验证概念、试用设计选项、发现更多的问题和可能的解决方法。



①建立开发原型的目标。这些可能是用户界面的原型、验证系统功能需求，或对管理者证明应用的可行性。同一个原型不能满足所有的目标。假如有遗漏的目标没有言明，则管理者或最终用户可能会误解原型的功能。因而原型的开发可能没有发挥预期的效果。

定义原型功能

②定义原型功能。决定什么要放入原型系统中，或许更重要的是，什么不要放入原型系统中。为了减少原型开发的成本并缩短交付时间，可能要在原型中放弃一些功能。

③原型开发。可能会放类似于响应时间、内存利用率等非功能的需求。错误处理和错误管理可能被忽略，除非原型的目标是构建用户界面。程序的可依赖性和质量方面的标准可能会降低。

④原型评估。这个阶段必须安排用户培训，应根据原型的目标制订一个评估计划。用户需要一段时间来适应新系统并习惯于正常的使用模式。一旦他们自然地使用了系统，他们将发现错误的和被遗漏的需求。

三、应对变更过程模型

1、原型开发（原型开发范型）

□ 软件原型的作用：

- 在需求工程过程中，原型有助于启发和验证系统需求。
- 原型可用于探索特定软件的解决方案，并支持用户接口设计。
- 开发原型时，可能会发现已提出需求的错误和遗漏。

□ 原型开发的问题：

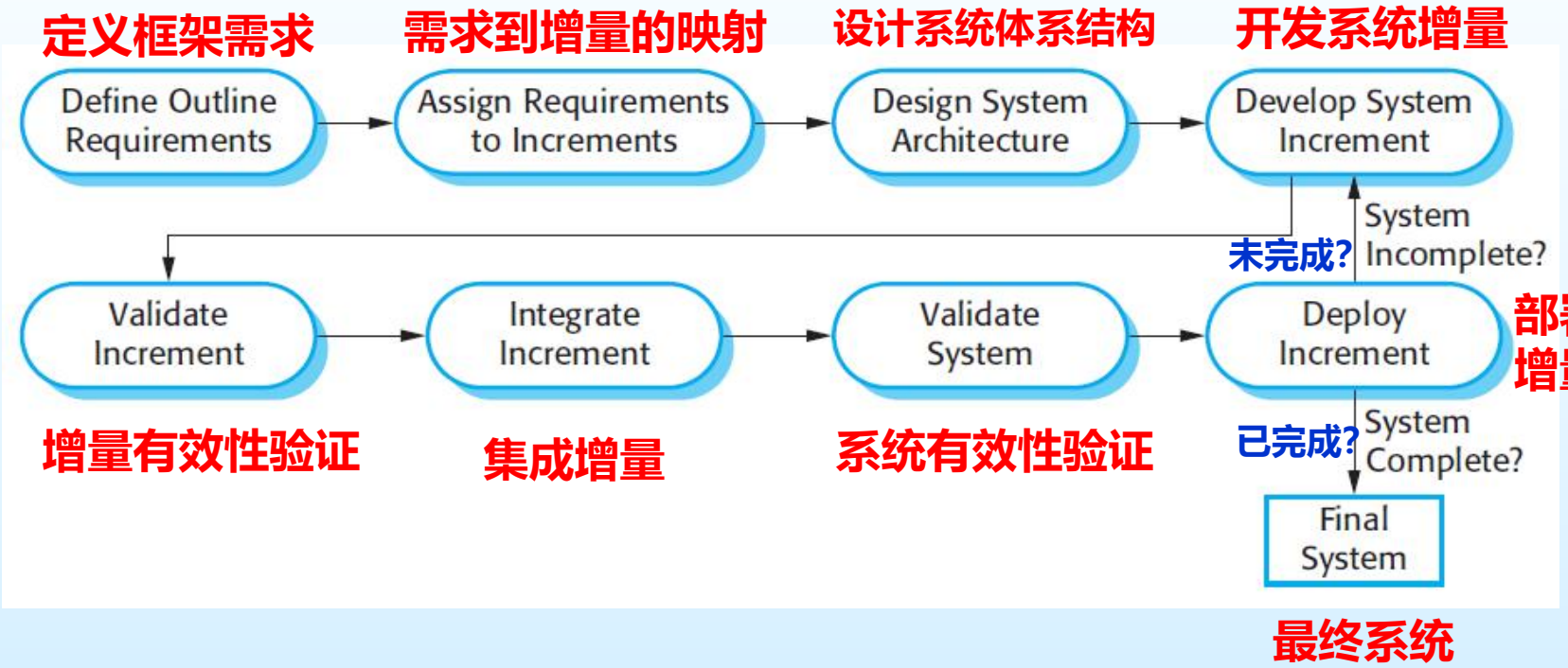
- 不可能调整原型以满足非功能性的要求，如性能、安全性、鲁棒性和可靠性需求。这些都会在原型开发时被忽略。
- 开发过程中的快速更改必然意味着原型是没有文档的。唯一的设计描述就是原型的代码。这不利于长期的维护。
- 原型开发过程中的变更可能会破坏系统的结构。系统的维护将会很困难，而且很昂贵。
- 在原型开发中机构的质量标准通常是被放松了的。

许多原型是临时系统并且会被废弃！

三、应对变更过程模型

2、增量式交付

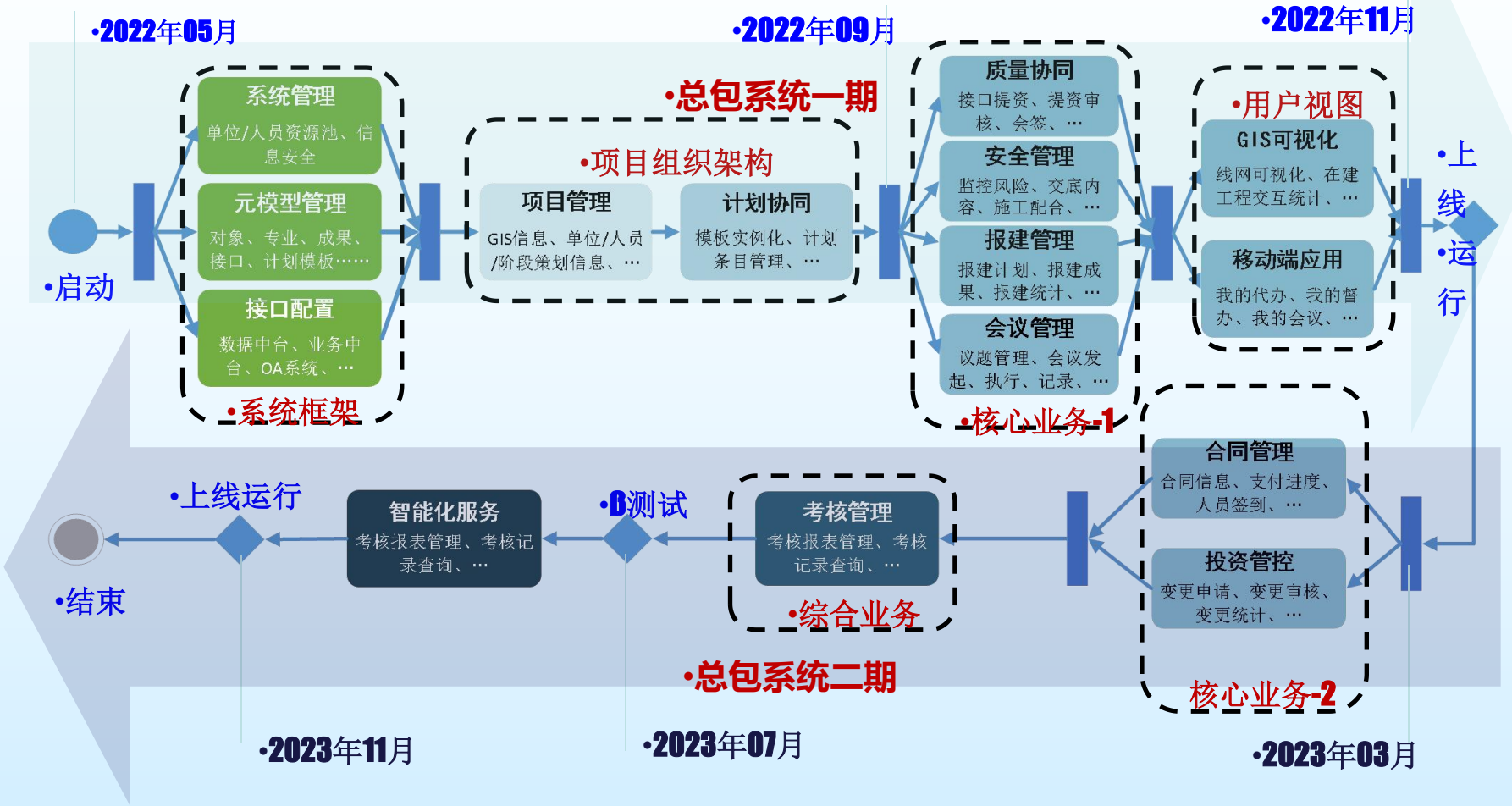
增量式交付是软件开发的一种方法，所开发的某些增量逐步交付给用户，并在用户的操作环境中运行。在增量式交付过程中，客户需要指明服务的重要级别，具有最高优先权的服务首先被交付。



三、应对变更过程模型

2、增量式交付—案例广铁总体总包系统

•依据模块间依赖度及业务协同需求的紧迫度，分阶段实施



三、应对变更过程模型

2、增量式交付

□ 增量式交付的好处：

- **客户可以将早期的增量作为原型，从中获得对后面系统增量的需求经验。**不像原型过程那样，这里的这些增量都是真正系统的一部分，因此，当系统全部完成时，用户不需要再重新学习。
- **客户无需等到整个系统的实现才能从中获益。**第一个增量会满足他们大多数关键的需求,因此，软件马上就能使用。
- 这一过程保持了增量式开发的优点，那就是相对来讲**变更能较容易地嵌入到系统中**。
- 因为具有最高优先权的服务被首先交付，而后面的增量也不断被集成进来，这就使得**最重要的系统服务肯定接受了最多的测试**。这也就意味着在系统的最重要的部分，客户不太可能遇到失败。

三、应对变更过程模型

2、增量式交付

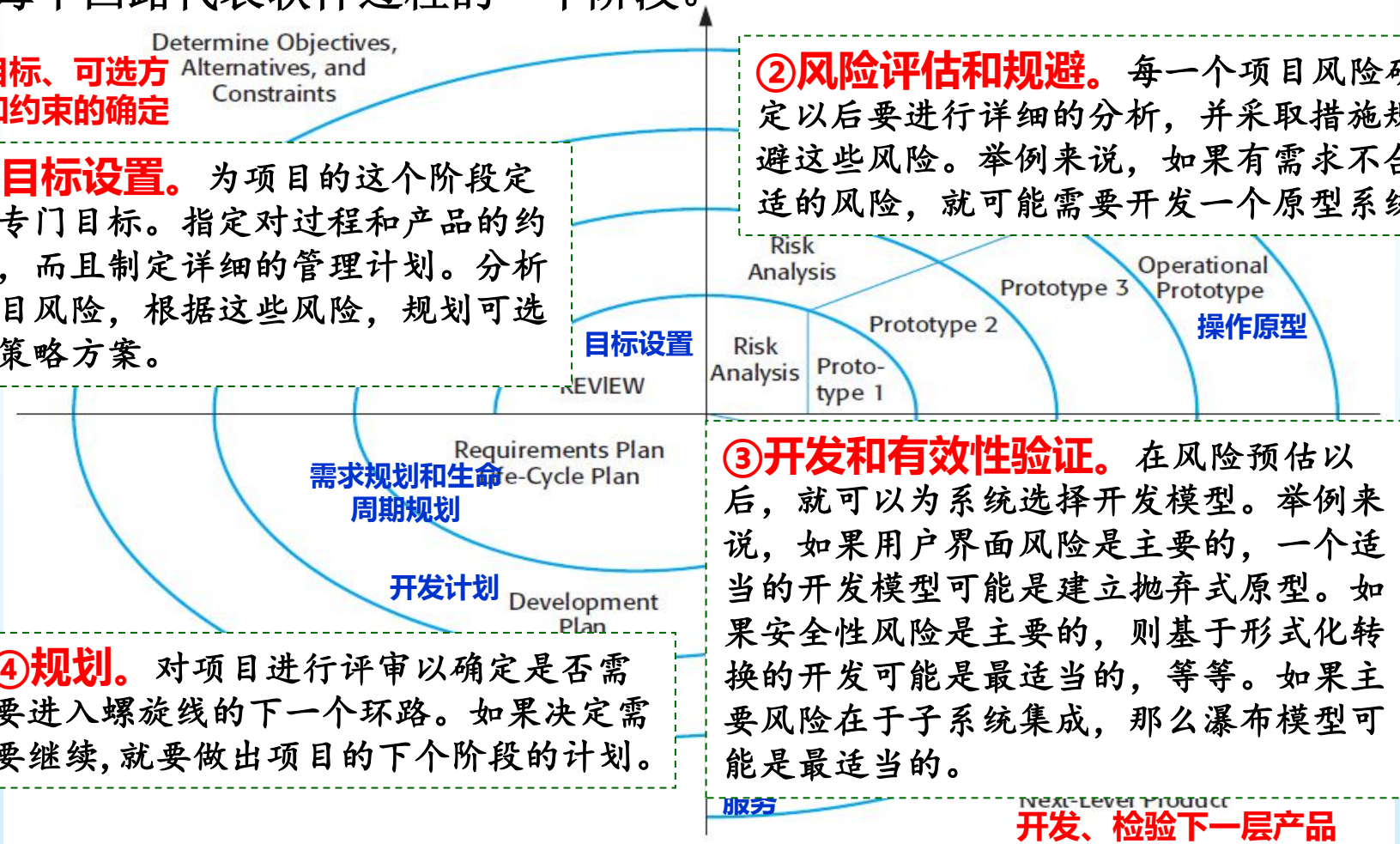
□ 增量式交付的缺点：

- 大多数系统需要一组基本设施，这些基本设施会被系统不同部分所使用。因为需求的详细定义是当增量将要被开发时才完成的，所以就**很难确定全体增量所需要的公用设施**。
- **当所开发的是一个替换系统时，迭代开发也是很困难的**。因为用户想要旧系统的所有功能，往往不愿意尝试一个不完整的新系统。因此，获取用户反馈是困难的。
- **软件描述和软件本身一起开发是迭代过程的本质**。然而，这与许多机构的采购模型是相冲突的，因为完整的系统描述往往是系统开发合同的一部分。在增量方法中，直到最后的增量描述完成，才会有完整的系统描述。这就需要一种新形式的合同，像政府机构这样的大客户将很难接受这样的合同。

三、应对变更过程模型

3、Boehm（勃姆）螺旋模型

风险驱动的软件过程框架（螺旋模型）最初由Boehm（1988）提出，每个回路代表软件过程的一个阶段。



①目标设置。为项目的这个阶段定义专门目标。指定对过程和产品的约束，而且制定详细的管理计划。分析项目风险，根据这些风险，规划可选的策略方案。

②风险评估和规避。每一个项目风险确定以后要进行详细的分析，并采取措施规避这些风险。举例来说，如果有需求不合适的风险，就可能需要开发一个原型系统。

③开发和有效性验证。在风险预估以后，就可以为系统选择开发模型。举例来说，如果用户界面风险是主要的，一个适当的开发模型可能是建立抛弃式原型。如果安全性风险是主要的，则基于形式化转换的开发可能是最适当的，等等。如果主要风险在于子系统集成，那么瀑布模型可能是最适当的。

④规划。对项目进行评审以确定是否需要进入螺旋线的下一个环路。如果决定需要继续，就要做出项目的下个阶段的计划。

三、应对变更过程模型

3、Boehm（勃姆）螺旋模型

其中：

① 评估和风险分析阶段都可作出一个决策：项目是否继续。

② 螺旋循环的次数指示了已消耗的资源；

③在规划阶段、风险分析阶段和开发阶段均进行需求规约活动；

④在早期螺旋循环中，为了为最终的实现给出一些指导性决策，经常使用原型构造；

⑤设计和实现活动一般是在开发阶段进行；

⑥ 验证和确认活动在开发阶段和评估阶段进行；

三、应对变更过程模型

3、Boehm（勃姆）螺旋模型

关于螺旋模型的几点说明：

（1）该模型关注解决问题的基本步骤：

♦标识问题；

♦标识一些可选方案，选择一个最佳方案；

♦遵循动作步骤，并实施后续工作。

其中只要完成了开发的一个迭代，开发的另一个迭代就开始。

（2）螺旋模型的一个特征是，实际上只有一个迭代过程真正开发可交付的软件。因此，如果

♦项目的开发风险很大，或

♦客户不能确定系统需求，在更广泛的意义上来讲，还包括系统或系统类型的要求，这时螺旋模型就是一个好的生存周期模型。

三、应对变更过程模型

3、Boehm（勃姆）螺旋模型

(3) 与其它模型的关系

①螺旋模型也使用瀑布模型作为一个嵌入的过程-即分析、设计、编码、实现和维护的瀑布过程，是螺旋一周的组成部分。

②尽管螺旋模型和一些迭代模型在框架和全局体系结构方面是等同的，但所关注的阶段以及它们的活动是不同的。

具体地说：❶标识客户想要的是一个什么样的系统；❷确定风险和效益的可选路线；❸选择最优方案；❹开发系统；❺评估完成情况等；❻重新开始。

即 螺旋模型扩展了增量模型的管理任务范围。而增量模型是基于以下假定：需求是最基本的、并且是唯一的风险源。而在螺旋模型中，决策和降低风险的空间是相当广泛的。

四、软件过程模型比较和应用

1、软件过程模型的比较

序号	模型名称	优点	缺点	适用场合
1	瀑布模型	1、线性模型，阶段间具有顺序性依赖性 2、文档驱动，每个阶段都有可交付的产品 3、每个阶段结束前进行审查，保证质量	1、早期错误要等到后期测试才能发现，增加了开发的风险； 2、大量文档极大地增加工作量 3、无法适应需求不明确和需求的变化	需求明确且稳定，技术成熟、工程管理严格的场合，如军工、航天、医疗
2	增量模型	1、软件作为一系列增量开发和交付 2、能看到软件的中间产品，降低开发风险 3、逐步交付，软件开发能够较好地适应需求的变化 4、软件能更早投入市场 5、开放式体系结构便于维护	1、软件必须具备开放式体系结构 2、易退化为边做边改的方式	软件开发中需求可能发生变化、具有较大风险、或者希望尽早进入市场的项目

四、软件过程模型比较和应用

1、软件过程模型的比较

序号	模型名称	优点	缺点	适用场合
3	螺旋模型	1、开发活动和风险管理结合； 2、开发分为若干提迭代，通过原型化降低或消除风险； 3、原型的进化贯穿整个开发周期，强调原型的可扩充性和可修改性，有助于软件的适用能力； 4、支持用户需求的动态变化； 5、为及时调整管理决策提供方便，进而降低开发风险。	1、采用螺旋模型需要具有相当丰富的风险评估经验和专门知识，在风险较大的项目开发中，如果未能够及时标识风险，势必造成重大损失； 2、过多的迭代次数增加开发成本，延迟提交时间。	需求不明确或者需求可能发生变化的大型复杂的软件系统
4	原型模型	1、通过建立原型明确并完善需求或者研究技术方案的可行性 2、减少需求不明确或者方案不可行带来的风险	1、构造原型采用的技术和工具不一定主流 2、快速建立起来的系统加上连续的修改可能导致质量低下	系统需求不明确或技术方案或算法可行性不明确
5	基于组件的开发模型	1、软件复用思想； 2、降低开发成本和风险； 3、加快开发速度，提高质量	商业构建不能修改，会导致需求的修改，进而导致系统不能完全符合要求	系统之间有共性的情况

四、软件过程模型比较和应用

2、软件过程模型的应用

□ 案例1：医疗设备控制软件

案例分析：

—需求明确且稳定
—可靠性和安全性要求极高
—对软件错误和故障的控制和跟踪能力强
—需要对软件开发过程严格控制
—需要大量严格的文档

→ 模型选择：瀑布模型

□ 案例2：校园一卡通系统

案例分析：

—包含若干相对独立的功能
—系统具体需求不明确且会发生变化
—系统需要具有可扩展性
—用户需要熟悉和适应新的系统
—项目复杂程度中等、风险较低
—产品和文档的再使用率较高

→ 模型选择：增量模型

四、软件过程模型比较和应用

2、软件过程模型的应用

□ 案例3：企业管理ERP系统

案例分析：

—包括销售、库存、生产、财务、物流、人力资源等部分

—在系统实施过程中不同的企业具有一定的那个的差异

—企业原来已有一定的组件模块积累

模型选择：**基于组件的开发模型**

□ 案例4：连锁店物流管理系统

案例分析：

—客户对需求不是很明确，无法一次性把需求完整提出

—各家连锁店需求不同

—开发人员有限

模型选择：**增量模型+基于组件的开发模型**

本章要点

本章要点

- 软件过程是产生一个软件系统的一系列活动。软件过程模型是这些过程的抽象表示。
- 一般过程模型描述软件过程的组成。一般过程模型实例包括**瀑布模型、增量式开发、面向复用的开发**。
- **需求工程**是开发软件描述的过程。描述的目的是向开发者传达客户方对系统的需要。
- **设计和实现过程**是将需求描述转换为一个可运行的软件系统的过程。系统化的设计方法用来完成这个转换。
- **软件有效性验证**是检查系统是否与它的描述相一致，以及是否符合系统用户的真正需要的过程。
- **软件进化**是修改已存在的软件系统以适应用户新的需求的过程。变更是一个持续的过程，软件必须在变更过程中保持可用。
- 过程应包含应对**变更**的活动。这可能包含一个原型构造阶段，以帮助避免在需求和设计上的错误决定。过程应该适应迭代开发和交付，这样变更时就不会对整个系统带来干扰。